



innovarsity
learn to innovate. lead to succeed.



NIELIT

skillbank

Integrate Experience Knowledge & skill



NumPy

PYTHON PROGRAMMING

PRESENTATION BY SRINIVASA SAI CHAVA

MODULE 1 OF ESSENTIALS OF AI

ABOUT ME

- M.S. Artificial Intelligence student at Boston University



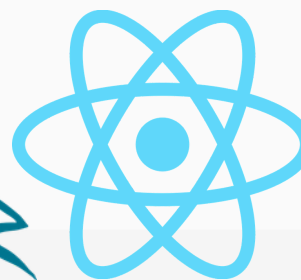
**BOSTON
UNIVERSITY**

- Building production-focused AI systems across ML/DL, NLP, Computer Vision, and RAG



**INDIAN INSTITUTE OF
INFORMATION TECHNOLOGY
DHARWAD**

- Delivering end-to-end full-stack AI deployment from research to production



- Focused on practical impact in accessibility, trust, and social good

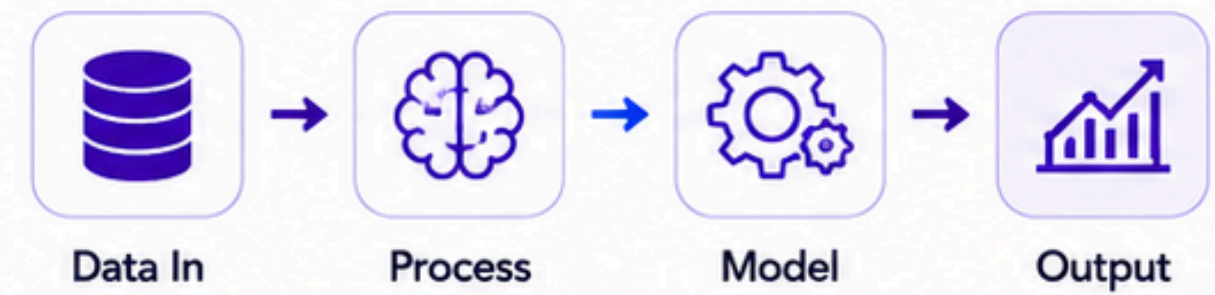
<https://github.com/essentials-of-artificial-intelligence>

GITHUB ORGANIZATION



<https://github.com/essentials-of-artificial-intelligence>

Functions, Errors, I/O, and OOP



- Transitioning from basic scripting to robust pipeline architecture.
- Today's focus: **Modularity** (Functions/Modules), **Resilience** (Exceptions/Files), and **State** (OOP).

Build smarter pipelines with modularity, resilience, and state management.

1 Modularity with Functions/Modules

Break your code into reusable, testable units.

Example

```
def clean_text(text):  
    return text.strip().lower()  
  
def word_count(text):  
    words = text.split()  
    return len(words)  
  
text = " Hello AI World! "  
print(clean_text(text))  
print(word_count(text))
```

Output: hello ai world! | 3



2 Resilience with Exceptions & I/O

Handle errors gracefully and work with files safely.

Example

```
try:  
    with open("data.txt", "r") as f:  
        content = f.read()  
        value = int(content)  
except FileNotFoundError:  
    print("File not found!")  
except ValueError:  
    print("Invalid number in file!")  
else:  
    print(f"Value: {value}")  
finally:  
    print("Done.")
```

Output: Value: 42 Done.



3 State with OOP

Model real-world entities and manage state through classes.

Example

```
class ModelTracker:  
    def __init__(self, name):  
        self.name = name  
        self.runs = 0  
  
    def log_run(self, metric):  
        self.runs += 1  
        print(f"{self.name} | Run {self.runs} | "  
              f"Metric: {metric:.4f}")  
  
tracker = ModelTracker("SentimentModel")  
tracker.log_run(0.8123)  
tracker.log_run(0.9231)
```

Output:

SentimentModel | Run 1 | Metric: 0.8123
SentimentModel | Run 2 | Metric: 0.9231



Putting It Together: Mini Pipeline Example



Key Takeaway

- Functions** = Reusability
- Errors & I/O** = Robustness
- OOP** = Managing State & Structure

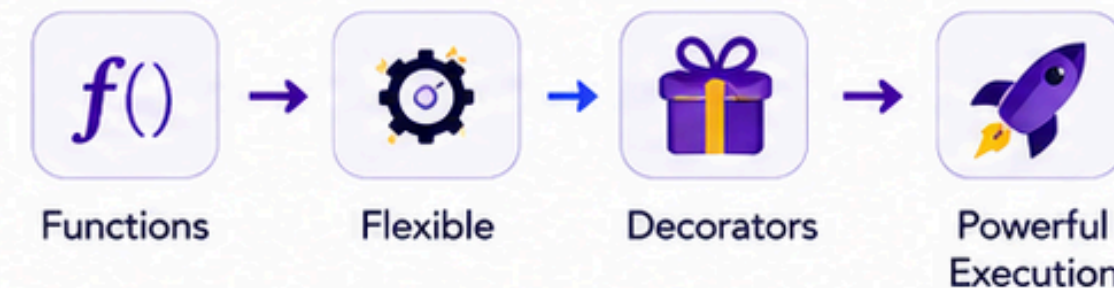


These building blocks help you design reliable, maintainable, and scalable AI pipelines.



Advanced Functions & Decorators

Flexible Execution



f(x) Functions are **first-class objects** (can be passed as arguments).

args** *kwargs** (tuples) and ****kwargs** (dictionaries) allow dynamic parameter counts.

</> **Decorators:** Wrap functions to modify behavior (e.g., **@time_it** for training loops).

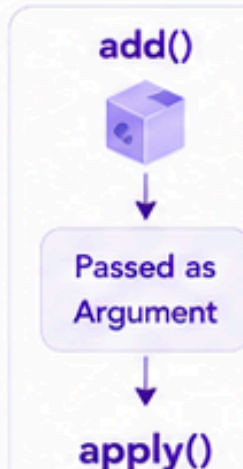


1 Functions are First-Class Objects

Functions can be assigned to variables, passed as arguments, and returned.

Example

```
def add(x, y):  
    return x + y  
  
def apply(func, a, b):  
    return func(a, b)  
  
result = apply(add, 5, 7)  
print(result) # 12
```



Output: 12

2 *args and **kwargs

Handle variable number of positional and keyword arguments.

Example

```
def info(*args, **kwargs):  
    print("Positional:", args)  
    print("Keyword:", kwargs)  
  
info(1, 2, 3, name="AI",  
    topic="Python", level="Pro")
```

***args**
(tuple)
(1, 2, 3)

****kwargs**
(dict)
{'name': 'AI',
'topic': 'Python',
'level': 'Pro'}

Output:

Positional: (1, 2, 3)
Keyword: {'name': 'AI', 'topic': 'Python', 'level': 'Pro'}



3 Decorators: Wrap & Enhance Behavior

Decorators wrap functions to add functionality without changing the core code.

Example: @time_it

```
import time  
  
def time_it(func):  
    def wrapper(*args, **kwargs):  
        start = time.time()  
        result = func(*args, **kwargs)  
        end = time.time()  
        print(f"⌚ {func.__name__} took {end-start:.4f}s")  
        return result  
    return wrapper  
  
@time_it  
def train(epochs):  
    time.sleep(1.2) # simulate training  
    return "Training completed ✅"
```

```
train(3)
```

Output:

⌚ train took 1.2003s
'Training completed ✅'



Key Takeaway



Functions are first-class citizens – powerful & flexible.



Handle any number of inputs with ease.



Decorators add superpowers to your functions.



Master these to write clean, reusable, and production-ready AI code!

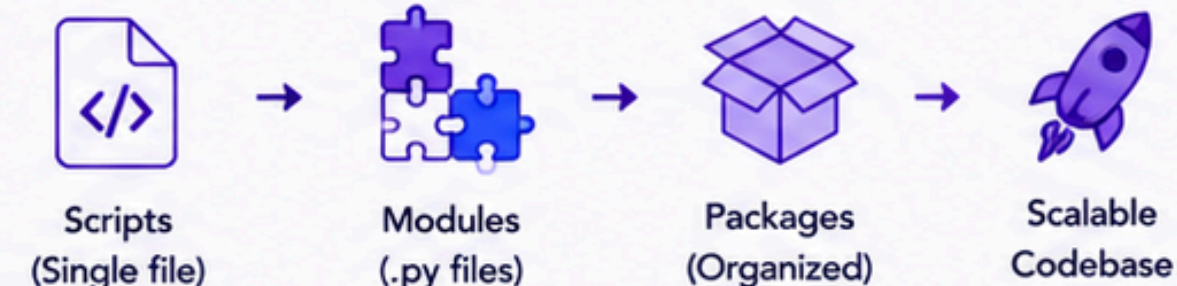


Modules and The Standard Library

Organizing Codebases

- Modules (.py files) separate data processing logic from model architecture.
- import** vs **from module import...**
- Critical built-ins for AI: **os** (directory management) and **math/random** (weight initialization).

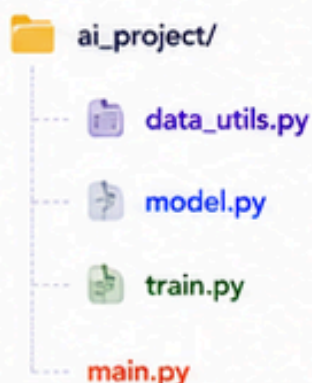
From Scripts to Scalable Systems



1 What is a Module?

A module is a .py file containing reusable code (functions, classes, variables).

Project Structure



data_utils.py

```
def load_data(path):  
    with open(path, 'r') as f:  
        return [line.strip() for line in f]  
  
def preprocess(data):  
    return [d.lower() for d in data]
```

main.py

```
import data_utils  
raw = data_utils.load_data('data.txt')  
clean = data_utils.preprocess(raw)  
print(clean[:3])
```



Modules promote separation of concerns and code reusability.

2 import vs from module import...

Two common ways to bring module code into your program.

import module

```
import math  
  
x = math.sqrt(16)  
y = math.pi
```

Access using
module name.

from module import...

```
from math import sqrt, pi  
  
x = sqrt(16)  
y = pi
```

Access directly
without module prefix.



Use **import module** for clarity in larger projects.
Use **from module import ...** for specific items to keep code concise.

3 Essential Standard Library for AI

Powerful built-in modules that every AI developer should know.

os – Directory Management

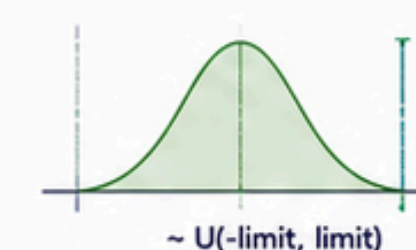
```
import os  
os.makedirs('runs/exp1', exist_ok=True)  
files = os.listdir('data/')  
print(files)  
print(os.path.join('data', 'train.csv'))
```



- ✓ Create directories, list files, and work with file paths.

math / random – Math & Initialization

```
import math, random  
fan_in = 128  
limit = math.sqrt(6 / fan_in)  
w = [random.uniform(-limit, limit)  
      for _ in range(fan_in)]  
print(f"Weight[0]: {w[0]:.4f}")
```



- ✓ Math for calculations, random for initialization.



Key Takeaway



Modules keep projects organized, reusable, and maintainable.



Choose the right import style to balance clarity and convenience.



Built-in libraries like **os**, **math**, and **random** are essential for AI workflows.



Exception Handling & Custom Errors

Resilient Pipelines

-  **try** (attempt code), **except** (catch error), **finally** (execute regardless, free up memory).
-  Catch specific errors (**KeyError**, **FileNotFoundError**) instead of blanket exceptions.
-  Use **raise** to build custom constraints (e.g., **InvalidGradeError**).

The Error Handling Pipeline



1 try, except, finally

Use **try** to attempt code, **except** to handle errors, and **finally** to clean up resources.

Example

```
def divide(a, b):  
    try:  
        result = a / b  
        return result  
    except ZeroDivisionError:  
        print("Cannot divide by zero!")  
    finally:  
        print("Cleaning up resources...")  
  
print(divide(10, 2))  
print(divide(5, 0))
```

Output:

```
5.0  
Cleaning up  
resources...  
  
Cannot divide by  
zero!  
Cleaning up  
resources...  
None
```



finally runs no matter what—perfect for closing files, connections, or freeing memory.

2 Catch Specific Errors

Catching specific exceptions helps you handle different failures appropriately.

Example

```
data = {"name": "Alice", "age": 25}  
  
try:  
    age = data["agee"] # Key does not exist  
    with open("data.txt", "r") as f:  
        content = f.read()  
except KeyError as e:  
    print(f"Missing key: {e}")  
except FileNotFoundError as e:  
    print(f"File not found: {e}")  
except Exception as e:  
    print(f"Unexpected error: {e}")
```

Output:

```
Missing key: 'agee'
```

★ Best Practice

Be specific so you can fix or recover intelligently.

Avoid:

except:
pass ❌

Prefer:

except SpecificError
as e: ✅



Specific errors → Better messages → Easier debugging → More reliable pipelines.

3 Custom Errors with raise

Use **raise** to enforce your own rules and create meaningful constraints.

Example

```
class InvalidGradeError(Exception):  
    """Raised when grade is outside valid range."""  
    pass  
  
def validate_grade(grade):  
    if not (0 <= grade <= 100):  
        raise InvalidGradeError(f"Invalid grade: {grade}")  
    return "Valid grade!"  
  
try:  
    print(validate_grade(95))  
    print(validate_grade(150))  
except InvalidGradeError as e:  
    print(f"Error: {e}")
```

Output:

```
Valid grade!  
Error: Invalid grade: 150
```

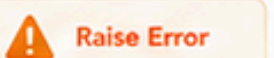


Custom errors make your code self-validating and easier for others to understand.

How raise Works



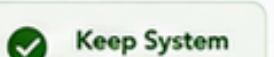
Check Condition



Raise Error



Stop & Handle



Keep System Safe



Key Takeaway



Use **try, except, finally** to build resilient and fault-tolerant code.



Catch specific exceptions (**KeyError**, **FileNotFoundError**) for precise handling.



Use **raise** to enforce business rules and create clean, reusable constraints.



Great AI pipelines don't just run fast—they **fail smart!**



File I/O & Context Managers

Managing Data Streams

 **open(file, mode):** Modes include **r** (read), **w** (write), **a** (append).

 **Context Managers:** The **with** statement safely handles memory and automatically closes files.

 Prevents **OS lockups** when loading massive text corpora.

The Flow of File Handling



1 File Modes with open(file, mode)

Different modes control how we access files.

Mode	Description	Example Use
r	Read (default). File must exist.	Open a config file
w	Write. Creates new file or truncates existing.	Save model logs
a	Append. Writes at the end of the file.	Append new results
r+	Read & Write.	Update in-place
b	Binary mode.	Images, models, bytes
t	Text mode (default).	Text data

Example

```
with open('data.txt', 'r') as f:
    text = f.read()
    print(text[:60])
```

Output (Preview)

The quick brown fox jumps over
the lazy dog...



Use **'r'** for reading, **'w'** for overwriting, **'a'** for appending. Choose wisely!

2 Context Managers with with

The **with** statement ensures files are properly closed—even if errors occur.

✓ Good: Using with (Safe)

```
def count_lines(path):
    with open(path, 'r') as f:
        lines = f.readlines()
    return len(lines)
```

- ✓ File is automatically closed.
- ✓ Memory is released.

Output:
125430

✗ Risky: Without with

```
def count_lines_bad(path):
    f = open(path, 'r')
    lines = f.readlines()
    # If an error happens before
    # f.close(), the file stays open!
    return len(lines)
```

- ✗ File may not close on error.
- ✗ Can cause memory leaks and OS lockups.

Output:
125430 (...but unsafe!)



with handles setup and teardown for you.
Cleaner code, fewer bugs, safer pipelines.

3 Handling Large Files (Prevent Lockups)

Stream big files line-by-line to save memory.

Example

```
def process_corpus(path):
    line_count = 0
    word_count = 0
    with open(path, 'r', encoding='utf-8') as f:
        for line in f:
            # Reads line by line
            line_count += 1
            word_count += len(line.split())
        if line_count % 100000 == 0:
            print(f"Processed (line_count) lines...")
    return line_count, word_count
```

Output (Sample)

```
Processed 100000 lines...
Processed 200000 lines...
...
(5234120, 89234125) # (lines, words)
```

Why it matters?



Key Takeaway



open(file, mode) gives you flexible control over data.



with statement automatically closes files and frees resources.



Safe file handling prevents lockups and keeps pipelines running smoothly.



Good I/O habits today save hours of debugging tomorrow!



Object-Oriented Programming (OOP) Part 1

State and Behavior

-  **Class:** The blueprint. **Object:** The instance loaded in memory.
-  The `__init__` dunder method initializes internal variables (**state**).
-  Perfect for defining independent **autograders** or **data preprocessors**.

From Blueprint to Working Object



1 Class vs Object

A class defines what an object has (attributes) and what it can do (methods).

Class (Blueprint)



Defines:

- Attributes (State)
- Methods (Behavior)

Example

```
class Student:
    pass # blueprint only

s1 = Student() # object (instance)
s2 = Student() # another object
```

Object (Instance)



Lives in memory with its own state.

Memory



Two independent instances



Classes are like cookie cutters; objects are the cookies—each with its own unique data!

2 `__init__` Initializes State

The `__init__` method runs automatically when an object is created, setting up its initial state.

Example

```
class Student:
    def __init__(self, name, score):
        self.name = name # state
        self.score = score # state

s1 = Student("Alice", 95)
s2 = Student("Bob", 88)
```

What happens?

1. Object created
2. `__init__` runs
3. State is set

Each object has its own data! ✨

Memory (State of Objects)



Changes in one object don't affect the other. 🔒



`__init__` is the **constructor**. It builds and initializes the object's internal state.

3 Behavior with Methods

Methods define what objects can do. They operate on the object's state.

Example

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def is_passing(self):
        return self.score >= 60

    def update_score(self, new_score):
        self.score = new_score

s1 = Student("Alice", 95)
print(s1.is_passing()) # True
s1.update_score(58)
print(s1.is_passing()) # False
```

Output:

True
False

How it works



Real-World Use Cases



Autograder

Each student submission is an object with methods to run tests and score.



Data Preprocessor

Each dataset/config is an object with methods to clean, transform and export.



Model Wrapper

Each model is an object with methods: `train()`, `predict()`, `evaluate()`.



Experiment Tracker

Each run/experiment is an object tracking params, metrics, and artifacts.



Key Takeaway



Class = Blueprint

Defines structure, attributes, and methods.



Object = Instance

Holds real data (state) in memory.



`__init__` = Constructor

Initializes state when the object is created.



OOP helps you build **modular**, **maintainable**, and **scalable** AI systems!



OOP Part 2 – Encapsulation & Dunder Methods

Protecting AI Architecture

- 🛡️ **Encapsulation:** Prefixing variables with `__` hides them from direct external modification (e.g., protecting neural weights).
- 📄 **The `__call__` method:** Allows objects to be executed like functions, which is the foundational design of **PyTorch/Keras layers**.

Why It Matters for AI Systems



Protect Internal State (Weights)



Prevent Accidental Modifications



Callable Objects (Layers/Models)



Reliable, Reusable AI Architectures



Scalable & Production-Ready

1 Encapsulation: Hide & Protect

Prefix variables with `__` (double underscore) to make them "private" to the class.

Example

```
class NeuralLayer:
    def __init__(self, in_dim, out_dim):
        self.__weights = self._init_weights(in_dim, out_dim)
        self.__bias = [0.0] * out_dim

    def _init_weights(self, in_dim, out_dim):
        # pretend weight init
        return [[0.01] * out_dim for _ in range(in_dim)]

    def forward(self, x):
        # use internal weights safely
        return [sum(wi * xi for wi, xi in zip(w_row, x)) + b
                for w_row, b in zip(self.__weights, self.__bias)]
```

Good: Protected

```
layer = NeuralLayer(3, 2)
layer.forward([1, 2, 3]) # OK
print(layer.__weights) # ❌
```

Bad: Direct Access

```
layer.__weights = [] # ❌
layer.__bias[0] = 999 # ❌
# Prevents accidental corruption of model weights!
```

How Encapsulation Works



💡 Encapsulation is like a vault: data is safe, and can only be accessed/changed through controlled methods.

2 The `__call__` Dunder Method

Makes an object callable like a function.
This is how layers/models are used in PyTorch/Keras!

Example

```
class LinearLayer:
    def __init__(self, in_dim, out_dim):
        self.__weights = self._init_weights(in_dim, out_dim)
        self.__bias = [0.0] * out_dim

    def __call__(self, x): # makes object callable!
        # forward pass
        return [sum(wi * xi for wi, xi in zip(w_row, x)) + b
                for w_row, b in zip(self.__weights, self.__bias)]
```

Use: Object as a Function

```
layer = LinearLayer(3, 2)
inp = [1.0, 2.0, 3.0]
out = layer(inp)
print(out) # __call__(inp)
```



Object (Layer)



PyTorch `nn.Module` and Keras layers use `__call__` under the hood. That's why you can do: `output = layer(input)`

3 Real-World AI Example

A custom layer with protected weights and callable interface.
Used just like built-in deep learning layers!

Example

```
import random

class Dense:
    def __init__(self, in_dim, out_dim):
        self.__weights = [[random.uniform(-0.1, 0.1) for _ in range(out_dim)]
                           for _ in range(in_dim)]
        self.__bias = [0.0] * out_dim

    def __call__(self, x):
        return [sum(xi * self.__weights[i][j] for i, xi in enumerate(x))
                + self.__bias[j] for j in range(len(self.__bias))]

# Use it like a function!
dense = Dense(3, 2)
y = dense([0.5, -1.2, 3.3])
print(y) # Works like nn.Linear!
```

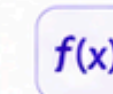
What You Get



Protected weights



Controlled access



Callable like a function



Clean, reusable AI components

Benefits



Safer models



Reusable components



Fewer bugs



Production ready

Common Dunder Methods You'll Use



`__init__(self, ...)`
Initialize object state



`__call__(self, ...)`
Make object callable



`__str__(self)`
Human-readable string



`__repr__(self)`
Developer-friendly string



`__len__(self)`
Return length/size



`__eq__(self, other)`
Equality comparison



Key Takeaway



Encapsulation protects your model's internal state (weights, biases, configs).



`__call__` lets your objects behave like functions — the core idea behind deep learning layers.



These principles power the design of modern AI frameworks like PyTorch and Keras.



OOP Part 3 – Inheritance & Polymorphism

Scalable Model Families

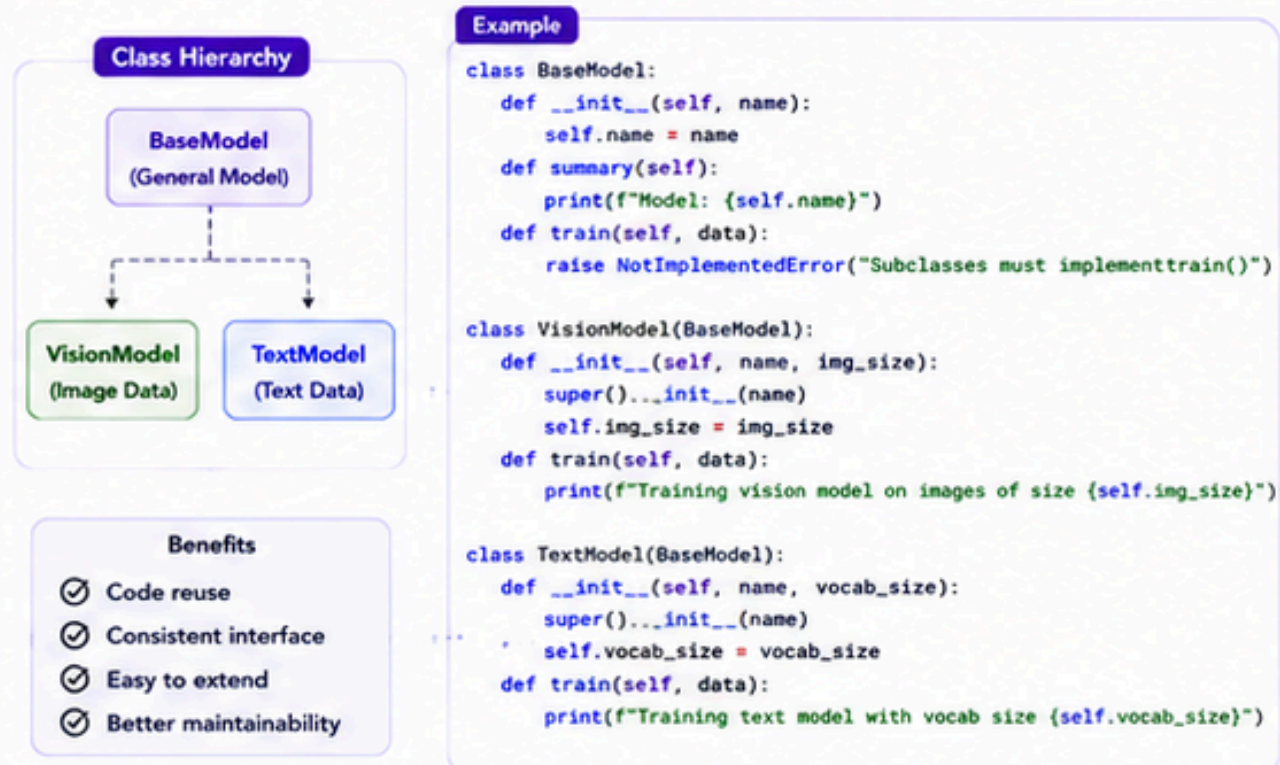
- Inheritance:** A child class inherits methods from a parent (e.g., `VisionModel` inherits from `BaseModel`).
- Polymorphism:** Child classes can override parent methods to behave differently based on data modality.

Why Inheritance & Polymorphism Matter



1 Inheritance: Reuse & Extend

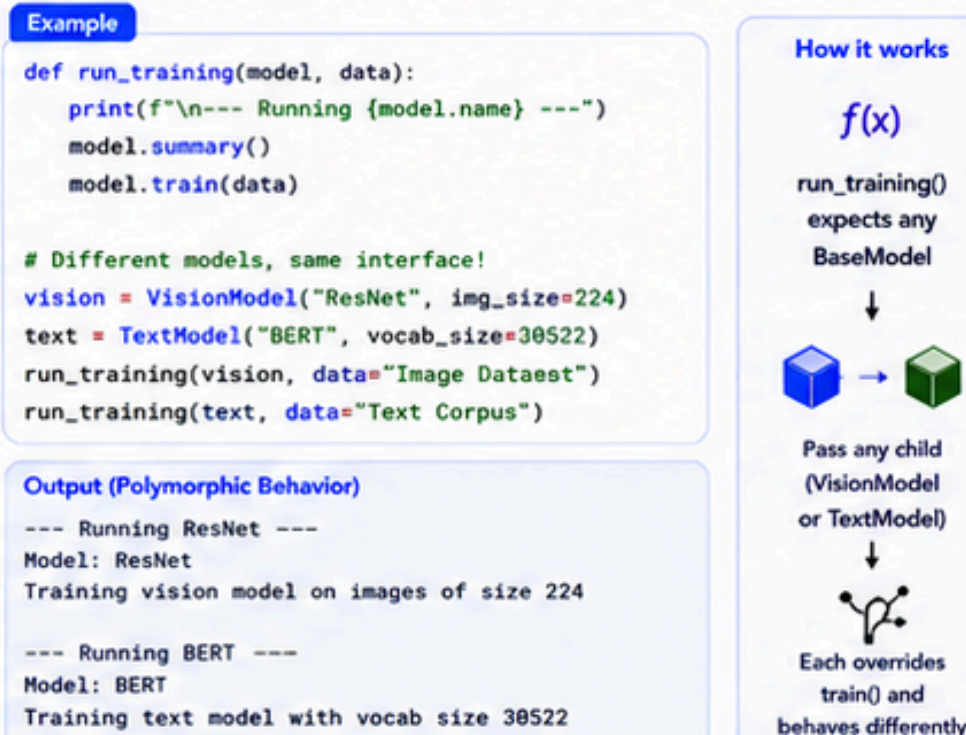
Child class inherits attributes and methods from parent.



★ Child classes (`VisionModel`, `TextModel`) inherit `summary()` and must implement `train()` in their own way.

2 Polymorphism: Same Interface, Different Behavior

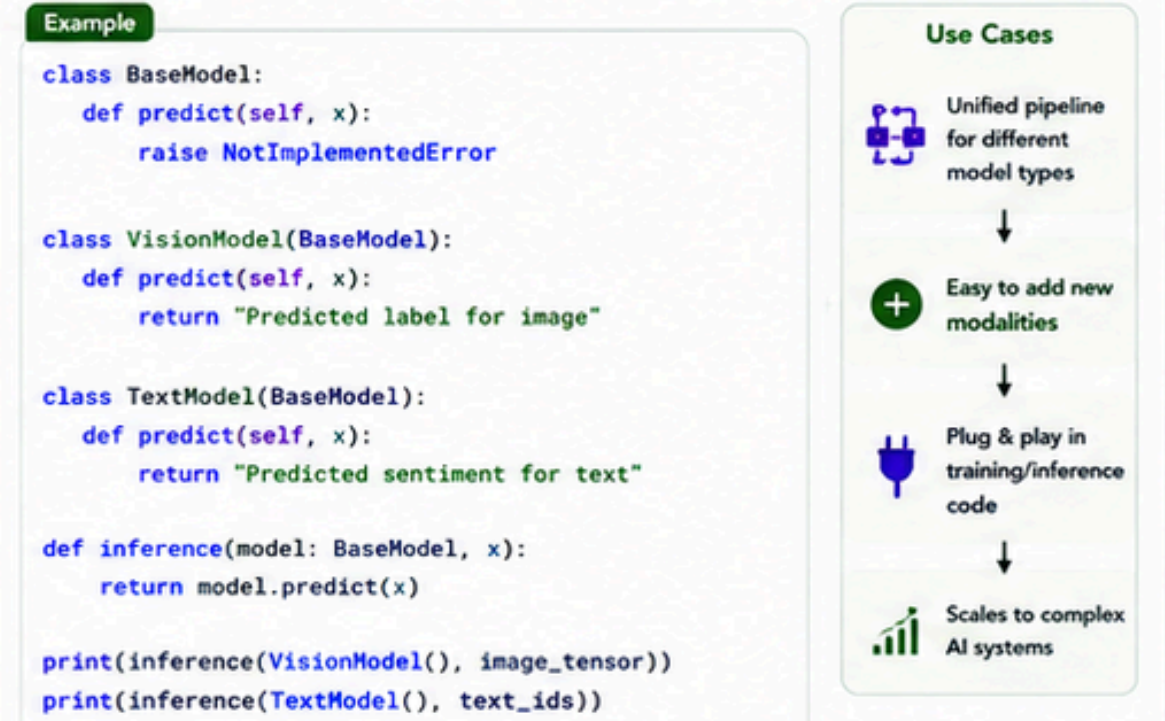
Different child classes override methods to provide behavior specific to their data modality.



🎯 Same method call (`train`) → different behavior based on the actual object passed. That's **polymorphism**!

3 Real-World AI Example

Build a family of models that share a common interface but specialize by modality.



✓ One interface (`predict`) → many implementations. Perfect for extensible, maintainable AI systems.

Common Pattern in AI



Key Takeaway



Inheritance promotes code reuse and clean hierarchies.



Polymorphism enables flexibility and extensibility.



Together, they help you build scalable model families for real-world AI.



Think: Define once, specialize many, use everywhere!

